Project Proposal: Reproducing and Extending LIME (Local Interpretable Model-agnostic Explanations)

Muhammad Hassan Shakoor Rana
Department of Computing, Macquarie University
muhammadhassanshakoor.rana@students.mq.edu.au

Abstract

This proposal outlines a plan to reproduce and extend the work of Ribeiro et al. (2016), which introduces LIME, a model-agnostic method for explaining individual predictions of any classifier. I describe the paper, explain its evaluation methodology and datasets, justify its selection as a novel project, and outline the new data, both existing and newly constructed, to which the method will be applied.

1 Description of Paper

Machine learning models, particularly complex ones such as random forests, gradient-boosted trees, or neural networks, are often used as "black boxes": they take an input and produce a prediction, but the internal reasoning behind that prediction is opaque even to the people who built the model. This creates a practical problem. If a model predicts that a patient has a disease, denies someone a loan, or flags an email as spam, the people affected by (or responsible for) that decision often need to know *why* the model made that call, both to trust the decision and to catch cases where the model is relying on spurious or biased reasoning.

The paper "Why Should I Trust You? Explaining the Predictions of Any Classifier" (Ribeiro et al., 2016), published at KDD 2016, addresses this problem by introducing **LIME** (Local Interpretable Model-agnostic Explanations), a technique for explaining individual predictions made by any classifier, regardless of its internal structure.

The key idea behind LIME is *local* approximation, as opposed to a *global* one. A global explanation would attempt to describe how a model behaves across its entire input space with a single, simple summary, but for a complex model, this is often not possible, since the model's true decision boundary may be highly non-linear and behave differently in different regions. A local explanation instead only attempts to describe the model's behaviour in the immediate neighbourhood of one specific input, where even a complex decision boundary can usually be well approximated by something much simpler, such as a straight line. LIME exploits this by focusing on explaining a *single prediction* at a time. It does this by:

1. Taking the specific input the model was asked to classify (e.g., one email, one image, or one row in a spreadsheet).

2. Generating a set of small, random perturbations of that input (e.g., removing some words from the email, hiding parts of the image, or slightly changing some feature values).

3. Asking the original black-box model to predict on each of these perturbed inputs.

4. Fitting a simple, interpretable model to these perturbed inputs and their predictions, weighted by how similar each perturbation is to the original input. This interpretable model is typically a *sparse* linear model, one in which most feature weights are forced to zero, so that only a small handful of features remain with a non-zero weight. Sparsity matters here because a linear model with hundreds of small, non-zero weights would be just as hard for a human to read as the original black box; restricting the explanation to only a few features is what makes it genuinely interpretable.

Because this interpretable model is only trained to approximate the black box in the local neighbourhood of one specific input, it does not need to be accurate everywhere, it just needs to capture the local decision boundary well enough to explain *that particular prediction*. The output is a short, human-readable explanation, such as a list of words (for text) or features (for tabular data) that pushed

the prediction toward or away from a given class, along with a weight indicating how influential each one was.

For example, if a spam classifier flags an email as spam, LIME might explain that prediction by highlighting the words "free," "winner," and "click here" as the words that most strongly pushed the model toward the "spam" label, even though the underlying classifier could be an opaque model like a neural network or gradient-boosted tree that doesn't natively expose this information.

The authors demonstrate LIME across three modalities, text classification, image classification, and tabular data, and show that it is **model-agnostic**: it treats the classifier purely as a black-box function that maps inputs to predictions (or predicted probabilities), so it can be applied to any classifier without needing access to its internal parameters or architecture. This generality, combined with the method's simplicity, is a major reason the paper has become one of the most cited works in the interpretable machine learning literature, with the accompanying `lime` Python package now a widely used, actively maintained tool.

2 Explanation of Evaluation

The LIME paper evaluates its method in two stages: first via **simulated experiments** (§5 of Ribeiro et al., 2016), where ground-truth answers can be constructed automatically, and then via **experiments with real human subjects** on Amazon Mechanical Turk (§6 of Ribeiro et al., 2016). Note that all section numbers of the form §n in this document refer to sections of the original paper being reproduced, not to sections of this proposal. Because the original paper draws on evaluation methodology that is not necessarily standard outside the interpretability literature, each part is explained below along with the datasets used and the concrete results reported.

2.1 Simulated Experiments: Faithfulness (Recall)

The core challenge in evaluating an explanation method is that, for real-world black-box models, the "ground truth" of which features are actually important is typically unknown, that is precisely the information LIME is trying to recover. To get around this, the authors first evaluate LIME on classifiers that are themselves interpretable by construction (sparse logistic regression and decision trees), trained on two sentiment-analysis datasets, **books** and **DVDs** (2,000 instances each, from product review data), with the maximum number of features any instance can use capped at 10. This gives a known "gold" set of important features for each prediction.

The metric used is **recall**: the fraction of the truly important (gold) features that an explanation method correctly recovers among its top-K highlighted features, averaged over all test instances.

Worked example. Suppose a sparse logistic regression model is trained such that, for a given instance, only 10 features are actually non-zero (the gold set). If an explanation method returns a ranked list of the top-10 features it believes are most influential, and it correctly matches 9 of the 10 truly non-zero features, then:

$$\text{Recall} = \frac{\text{true important features recovered}}{\text{total true important features}} = \frac{9}{10} = 0.9$$

The paper compares LIME against three baselines: **parzen** (explaining via gradients of a Parzen-window approximation of the classifier), **greedy** (iteratively removing the most class-contributing feature), and **random** (randomly selected features). The reported results (Figures 6–7 of the paper) show LIME consistently achieving over 90% recall on both datasets and both classifier types (92.1% and 97.0% on books for sparse LR and decision trees respectively; 90.2% and 97.8% on DVDs), substantially outperforming parzen (which ranges from roughly 61% to 89%), greedy (37%–64%), and random (13%–21%). This demonstrates that LIME's explanations are faithful to what the underlying interpretable models are actually doing.

2.2 Simulated Experiments: "Should I Trust This Prediction?"

The second simulated experiment tests whether explanations can help identify untrustworthy individual predictions, without yet involving real human subjects. The authors randomly select 25% of a model's features to be treated as "untrustworthy" (simulating features a user would not want to rely on, such as document headers), and construct an oracle label: a prediction is treated as genuinely "untrustworthy" if it changes when those untrustworthy features are removed from the instance, and "trustworthy" otherwise. A simulated user is then assumed to distrust a prediction if the untrustworthy features appear in that prediction's explana-

tion. Comparing each explanation method's simulated trust judgement against the oracle, the paper reports (Table 1 of Ribeiro et al., 2016) that LIME achieves an average trustworthiness F_1 of 94.5–96.7% across four classifier types (logistic regression, nearest neighbours, random forests, and SVM) on both the books and DVDs datasets, compared to 81.7–94.3% for parzen, 45.0–58.1% for greedy, and around 14–15% for random.

2.3 Simulated Experiments: "Can I Trust This Model?"

The third simulated experiment evaluates whether explanations can help select between two competing models with similar validation accuracy. The authors inject 10 artificial "noisy" features into random forest classifiers (again on the books and DVDs sentiment datasets), such that pairs of classifiers are constructed with near-identical validation accuracy but test accuracy differing by at least 5%, making it impossible to identify the better classifier from validation accuracy alone. A simulated user inspects B explained instances and marks noisy features as untrustworthy; the paper reports the accuracy of correctly identifying the better classifier as B varies (Figure 8 of Ribeiro et al., 2016), finding that LIME combined with the paper's proposed **submodular pick** procedure for selecting which instances to show a user (§4 of Ribeiro et al., 2016) outperforms all other method/pick combinations, particularly when only a small number of instances are shown.

2.4 Human Subject Experiments (Amazon Mechanical Turk)

Three real human-subject experiments are reported in §6 of Ribeiro et al. (2016), all using SVM classifiers trained on the **20 Newsgroups dataset** (Lang, 1995), restricted to the "Christianity" vs. "Atheism" subset. This subset is deliberately chosen because it contains features that do not generalise well (informative header information, author names), causing validation accuracy to considerably overestimate real-world performance. To measure real-world performance, the authors additionally constructed a new **religion dataset** of 819 webpages per class, gathered from the DMOZ directory and human-curated lists.

Can users select the best classifier? 100 Mechanical Turk users were shown side-by-side explanations (restricted to $K = 10$ words per explanation, $B = 6$ instances) for two competing classifiers: the original SVM trained on 20 Newsgroups (57.3% accuracy on the religion dataset, despite 94.0% held-out accuracy on the original 20 Newsgroups split), and a manually "cleaned" version with non-generalising features removed (69.0% accuracy on the religion dataset, 88.6% on 20 Newsgroups), a case specifically constructed so that trusting the higher held-out accuracy alone would lead to selecting the worse classifier. Users choosing based on LIME's explanations with the submodular pick procedure correctly identified the better classifier 89.0% of the time, compared to 75.0% with random pick, and 80.0% and 68.0% respectively for a greedy baseline (Figure 9 of Ribeiro et al., 2016).

Can non-experts improve a classifier? Users were shown explanations from the worse (uncleaned) 20 Newsgroups classifier and asked to mark words that should be removed from training as non-generalising. Across three rounds of iterative retraining (10 initial subjects, 250 classifiers total by the final round), the resulting classifiers' real-world accuracy (measured on the religion dataset) improved substantially, with LIME's submodular pick outperforming random pick, despite users being non-experts in machine learning with no access to the religion dataset itself (Figure 10 of Ribeiro et al., 2016). Each round took subjects an average of 3.6 minutes.

Do explanations lead to insights? (Husky vs. Wolf) In a striking illustrative experiment, the authors intentionally trained a "bad" logistic regression classifier to distinguish photos of wolves from Siberian huskies, using only 20 hand-selected training images such that every wolf photo had a snowy background and no husky photo did. On 60 additional test images, the resulting classifier was found to be predicting "wolf" whenever snow was present in the image, regardless of the actual animal. 27 graduate students (with prior machine learning coursework) were shown 10 example predictions, including one clear mistake (a husky photographed in snow, incorrectly predicted as "wolf"), and asked whether they trusted the classifier and why. Before seeing any LIME explanations, 10 of 27 subjects trusted the classifier, and only 12 of 27 identified snow as a possible feature the model was using. After being shown LIME's explanations (which visually highlight the snow region as the predictive feature, as in Figure 11 of the paper), only 3 of

27 subjects still trusted the classifier, and 25 of 27 correctly identified snow as the determining feature (Table 2 of the paper).

2.5 Datasets Used

In summary, the original paper's experiments draw on:

- **Books and DVDs sentiment datasets** (Blitzer et al., 2007) (2,000 instances each), used for all three simulated experiments (faithfulness/recall, prediction trust, and model trust).
- **20 Newsgroups** (Lang, 1995), restricted to the "Christianity vs. Atheism" subset, used for all three human-subject experiments and for the text-classification example in Section 3 of the paper (illustrating the SVM/"Posting" spurious-feature case).
- A newly constructed **religion dataset** (819 webpages per class), built by the authors themselves from the DMOZ directory and human-curated lists, used to measure real-world generalisation independent of the 20 Newsgroups validation split.
- **ImageNet-style natural images**, used with a pre-trained Inception network (Szegedy et al., 2015) for the image-classification explanation example in Section 3 of the paper (an image of a dog holding an acoustic guitar, misclassified with high confidence as "electric guitar").
- A small hand-curated set of 20 training images (and 60 test images) of wolves and huskies, constructed specifically for the Section 6.4 spurious-correlation experiment.

2.6 Could an LLM Judge Be Used to Evaluate LIME?

An **LLM-as-judge** is an evaluation approach in which a large language model is prompted to assess the quality of an output (e.g., a generated explanation) in place of, or alongside, a human evaluator, typically by scoring or comparing outputs against stated criteria. This fits evaluation tasks that are inherently *judgement-based* rather than tasks with a single objectively correct answer.

Assessed against LIME's own evaluation methodology (§2), an LLM judge is **not** suitable for every part of the original evaluation, but **is** plausibly applicable to part of it:

- **Not applicable: the recall/faithfulness experiments (§5.2 of Ribeiro et al., 2016).** These compare LIME's output against a mathematically defined ground truth (the known non-zero features of a sparse model). Recall is an objective, computable metric; an LLM judge would add no value here, since there is nothing subjective to weigh in on.
- **Plausibly applicable: the human-trust experiments (§5.3–5.4 and §6 of Ribeiro et al., 2016).** These fundamentally ask a human (via Mechanical Turk) to make a *subjective judgement*: does this explanation make the prediction seem trustworthy? Would I remove this word from training? These are exactly the kind of judgement-based questions LLM-as-judge approaches are designed for elsewhere in the literature (e.g., evaluating summary quality). An LLM could plausibly be prompted with a prediction, its LIME explanation, and a question analogous to those asked of the paper's Mechanical Turk annotators, with its response compared against the paper's reported human results.

This project will therefore explore LLM-as-judge as a **scalable complement to, not a replacement for, the human-subject trust experiments**. An LLM judge's output is not a ground-truth trust judgement any more than a human annotator's is, so its value lies in whether it *correlates with* the paper's human results, not in independently "proving" LIME works. LLM judges can exhibit systematic biases (e.g., preferring longer or more confidently worded explanations) that may not match genuine human trust formation; this project will treat the original paper's human-subject results as the reference point against which any LLM-judge results are compared, rather than assuming LLM judgements are correct by default. As this technique is only covered in detail later in the unit (Week 7 onward), the specific prompting methodology will be developed further as the project progresses.

3 Justification for Paper Choice

3.1 Quality of the Source Work

Ribeiro et al. (2016) was published at KDD 2016 (ACM SIGKDD International Conference on Knowledge Discovery and Data Mining), a CORE A* ranked venue and one of the top conferences in data mining. As of writing, Google

Scholar lists the paper's citation count in the tens of thousands, and the code repository (https://github.com/marcotcr/lime) shows approximately 12,100 stars and 1,850 forks, both indicating substantial reach and adoption within the machine learning community. Beyond raw citation count, LIME is notable for having shaped an entire sub-field: the SHAP method (Lundberg and Lee, 2017), now similarly prominent, explicitly positions itself relative to LIME's local-explanation framing.

3.2 Feasibility

As a basic requirement for a novel project, the associated code repository (https://github.com/marcotcr/lime) was cloned and executed successfully prior to submitting this proposal, on Python 3.11.9 with `lime` 0.2.0.1, scikit-learn, pandas, and numpy. The following was verified:

- The repository is well-organised, includes a test suite, and supports tabular, text, and image data via separate explainer modules (`lime_tabular`, `lime_text`, `lime_image`).
- The package installs cleanly via `pip`.
- The tabular explainer was run end-to-end: a Random Forest classifier was trained on the Iris dataset (100% test accuracy), and LIME was used to explain an individual prediction. The explanation correctly identified petal length as the dominant feature driving the classification of the *setosa* class (weight ≈ 0.47), consistent with known domain properties of this dataset.
- The text explainer was also tested on the "Christianity" vs. "Atheism" subset of the 20 Newsgroups dataset (Lang, 1995) used in the original paper, using a Naive Bayes classifier (1,079 training documents, 717 test documents), achieving an F_1 score of approximately 0.92. LIME's explanation for an individual prediction correctly surfaced email-header artefact words such as `Host`, `Posting`, and `NNTP` as influential features, in direct agreement with the spurious-feature behaviour reported by Ribeiro et al. (2016).

One environment-specific issue was encountered and resolved during setup: an SSL certificate verification error when downloading the 20 Newsgroups dataset on macOS, a known and well-documented Python/macOS configuration issue unrelated to the LIME codebase itself, resolved by installing the appropriate root certificates. No modification to the LIME package or its dependencies was required.

This package imposes no unusual compute requirements: all experiments above ran in well under a minute on CPU only, with no GPU or large-scale infrastructure required. This supports the feasibility of applying LIME to new datasets within the scope of this unit.

4 New Data

4.1 Existing Datasets

Five candidate existing datasets, not previously explained via LIME in the original paper, are proposed below. For each, a concrete feasibility assessment is given, covering size, format, access method, and an intended pairing with a classifier.

1. **Credit Card Fraud Detection** (Kaggle, `mlg-ulb/creditcardfraud`), approximately 285,000 transactions, 30 numerical features, with a binary fraud label (only 0.17% fraudulent, strongly imbalanced). A single CSV file, freely downloadable via Kaggle. A Random Forest or gradient-boosted classifier would be trained on a stratified split, with LIME's tabular explainer applied to fraud predictions. Feasibility is high: small enough to load fully into memory, with numerical features directly compatible with `LimeTabularExplainer`.

2. **IMDB Movie Reviews** (HuggingFace `datasets`, `stanfordnlp/imdb`), 50,000 movie reviews (25,000 train / 25,000 test), balanced across positive/negative sentiment labels, loading directly via `datasets.load_dataset("imdb")` in a single line. A TF-IDF + logistic regression or linear SVM pipeline (mirroring the classifier setup in the original paper's text experiments) would be trained, with LIME's text explainer highlighting words driving sentiment predictions, extending LIME beyond the topic classification setting of 20 Newsgroups into sentiment analysis. Feasibility is high: a standard HuggingFace-hosted dataset with no licensing barriers and modest compute requirements.

3. **Heart Disease UCI Dataset** (UCI Machine Learning Repository / also mirrored on Kaggle), 303 patient records with 14 attributes (e.g., age, cholesterol, resting blood pressure) and a binary presence/absence-of-disease label, loadable directly as a CSV with `pandas`. A logistic regression or small Random Forest classifier would be trained, with LIME explaining individual diagnosis predictions, directly relevant to the original paper's own motivating example of trust in medical decision support (Figure 1 of the paper). Feasibility is very high given the small size and lack of preprocessing complexity, though the small sample size is itself worth discussing as a limitation in the final report (e.g., higher variance in explanation stability across resampling).

4. **Toxic Comment Classification** (Kaggle, Jigsaw/Conversation AI toxic comment challenge dataset): approximately 160,000 Wikipedia talk-page comments across six toxicity categories, simplified to a binary "toxic vs. non-toxic" label for comparability with the paper's binary text setup. A TF-IDF-based linear classifier would be trained, with LIME explaining which words drove a toxic classification, relevant to moderation transparency. Feasibility is good, though the data is noisier than the others, requiring some cleaning before training.

5. **Adult / Census Income Dataset** (UCI Machine Learning Repository, also via scikit-learn's `fetch_openml`): approximately 48,000 records with 14 demographic and employment attributes, with a binary label indicating income above $50K/year, a classic fairness benchmark. A Random Forest or logistic regression classifier would be trained, with LIME testing whether explanations expose reliance on sensitive attributes (e.g., sex, race), relevant to surfacing ethically problematic feature reliance. Feasibility is high: a standard benchmark requiring only categorical encoding before use with `LimeTabularExplainer`.

Across all five datasets, feasibility is supported by three shared factors: each is a standard, freely available benchmark; each pairs with a lightweight, CPU-trainable classifier; and each is compatible with LIME's existing explainer modules with no codebase modification.

4.2 Newly Constructed Data

As an idea for newly constructed data, a small dataset of short product or service reviews will be collected and deliberately manipulated to contain an injected spurious correlation, mirroring the "Host"/"Posting"/"NNTP" spurious-feature behaviour observed in the feasibility experiment (§3). Rather than a single injected correlation, this will be structured as a controlled experiment across multiple correlation strengths, allowing a more precise test of LIME's sensitivity.

Concretely, the construction plan is:

1. Collect a modest set of short reviews (on the order of a few hundred), drawn from a publicly available source such as a review-platform export or an existing open review dataset, spanning both positive and negative sentiment.

2. Construct several versions of the dataset, each with a semantically meaningless artefact token injected into a different proportion of one class's reviews (100%, 75%, 50%, 25%, and a 0% control), producing a gradient of correlation strength rather than a single all-or-nothing manipulation.

3. Train a separate text classifier on each version, recording reliance on the artefact (e.g., probability drop when removed).

4. Apply LIME to predictions from each classifier, recording whether the artefact appears among top-ranked features at each correlation level.

5. Compare the correlation strength at which LIME surfaces the artefact against the strength at which the classifier becomes reliant on it, extending the trust-detection evaluation of Section 2 to a graded rather than binary setting.

This keeps the dataset small while producing a more rigorous, quantifiable test than a single injected-artefact condition. Some human judgement may be required in selecting artefact placements and verifying labels remain sensible; as annotation techniques are covered later in the unit, the precise protocol will be finalised at that stage.

References

John Blitzer, Mark Dredze, and Fernando Pereira. 2007. Biographies, bollywood, boom-boxes and blenders: Domain adaptation for sentiment classification. In *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Ken Lang. 1995. Newsweeder: Learning to filter netnews. In *Proceedings of the Twelfth International Conference on Machine Learning (ICML)*, pages 331–339.

Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1135–1144.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.